

A Lightweight Hallucination Detection Algorithm for the Reviewer Agent in a Code-Generation Multi-Agent System

Syed Muhammad Shajee Raza

ITMO University, Faculty of Digital Transformations, Saint-Petersburg, 2026

Abstract

Hallucinations, which are outputs that sound fluent but are actually contradictory to the given context, have been a major problem when large language models (LLMs) are used to power autonomous code-writing agents. This article introduces a simple hallucination detection method that serves as a Reviewer Agent inside a multi-agent system orchestrated by LangGraph. In fact, the detector does not even need to be trained, have access to model logits, or use an external verifier: it just makes one additional call to the reviewer LLM, checks the response against a Pydantic schema, and rates the output on a five-dimension rubric comprising Correctness, Edge Cases, Security, Code Quality, and Relevance. If the average score is lower than the acceptability threshold, the generator goes again with the Reviewer's feedback; this cycle lasts up to three times, at most.

Another point, the Hallucination Bridge, uses the same audit logic on the per-step action traces of an autonomous OpenHands coding agent, resulting in individual agent actions that can be verified forensically.

We run the system on 19 instances of SWE-bench Lite grouped into five controlled sweeps, taken from django, sympy, scikit-learn, and astropy. Condition B at an acceptance threshold of 0.85 achieves a mean relevance score of 0.926 ($\sigma = 0.044$, floor at 0.85) against a single-shot baseline of 0.878 ($\sigma = 0.145$), confirming that the refine loop acts as a hard score floor. With a weaker generator (qwen2.5-7b) the Reviewer adds $\Delta = +0.114$ in mean score, compared to $\Delta = +0.003$ when generator and judge share the same strong model, the Reviewer contributes most where it is needed most. Condition C at threshold 0.85 is bimodal: 14 of 19 instances score 0.86–1.0 while five regress to ≤ 0.40 , a failure mode traced to context explosion in long agentic runs.

1. Introduction

Large language models (LLMs) have quickly evolved from being chat-based tools to fully autonomous programming agents that can plan, execute terminal commands, and even commit patches without the assistance of humans at every step. Unfortunately, the ways they can fail have also increased with their capability, e.g. a single run of an agent using 30 different tool calls on a real repository may involve creating a function signature, misunderstanding a stack trace, or falsely claiming the tests have passed when in fact they have not. These examples are not glitches but are typical patterns that show up in evaluation data from SWE-bench and its derivatives [1].

Current detection techniques can be divided into two categories. Interventions during training (RLHF [2], DPO) not only require labeled data but also re-training. On the other hand, at inference time, detectors generate multiple outputs and check the consistency (SelfCheckGPT [3]), depend on model logits [4], or a third party verifier is needed. However, none of these methods are effectively compatible with a practical multi-agent setup which operates on ordinary hardware especially when free-tier APIs are used.

In this regard, we have made three important contributions. First, we come up with a Reviewer Agent that is a prompt-only hallucination detector based on rubric of the five dimensions and the output is Pydantic-validated, not requiring access to model internals. Second, we introduce the Hallucination Bridge, which uses the same rubric for not only textual outputs but also structured per-step action traces that are produced by an autonomous OpenHands agent [5]. Third, we present data from five controlled experimental sweeps over 19 SWE-bench Lite instances, where we expose the effects of different factors like threshold selection, judge-generator separation, and agent design.

2. Related Work

Maynez et al. [6] defined hallucination detection in natural language generation as either intrinsic hallucination (output contradicts the source) or extrinsic hallucination (output contains unsupported facts).

Ji et al. [7] covered a broader scope. Chen et al. [8] in the case of code generation found that just surface plausibility is not a good enough verification of correctness, which led to the use of execution-based evaluation. SelfCheckGPT [3] at zero-resource detection through consistency sampling but needs multiple inference passes and is not very compatible with the extension to action traces.

The LLM-as-a-judge paradigm [9] found out through an experiment that a strong LLM when prompted with an adequately structured rubric can mimic human preferences on various types of tasks.

Pan et al. [12] highlight the fact that self-correction improvements are reducing and are likely to demonstrate self-bias in cases where critic and generator have shared weights. Self-Refine [10] and Reflexion [11] proposed iterative critique and revise loops.

Yang et al. [2024] and OpenHands [5] offer frameworks for agent-based programmatic interactions in a repository but do not incorporate hallucination auditing of action traces. This paper addresses the issue by integrating a rubric-based reviewer into the orchestration graph and expanding audit coverage to the action sequences.

3. Methods

3.1 System Architecture

The system consists of LangGraph and once compiled, becomes a directed state machine parameterized with one of the variant identifiers (A, B, or C). All communication among the agents is carried out through a ChatState typed dictionary. The graph consists of eight nodes: Router, Supervisor, Researcher, Writer, Critiquer, Code Helper, Quiz Helper, and the new Reviewer Agent. In the case of Condition C, ChatState also includes five OpenHands-specific fields: agent status, backend id, step count, wall time, and the complete per-step action trace serialized as a list of dictionaries.

	ARCHITECTURE	REVIEWER BEHAVIOUR	REFINE LOOP	IDENTITY
A	Code-helper agent only	Single review, scoring only	—	Baseline
B	Code-helper + Reviewer	Full 5-dimension rubric	Up to 3 iterations if $C < 0.7$	Reviewer + Refine
C	OpenHands agent + Reviewer	Final answer + per-action trace	Up to 3 iterations if $C < 0.7$	Hallucination Bridge

Figure 1. Three-condition experimental design. Condition A is single-shot; Condition B adds a Reviewer-gated refine loop; Condition C replaces the Code Helper with an autonomous OpenHands agent.

3.2 The Lightweight Detection Algorithm

The Reviewer Agent enforces 3 design constraints. It’s prompt based, no fine-tuning, no logit access, and deployable against any OpenAI compatible endpoint. The output is validated against a Pydantic model, RelevancyDecision, with fields is_relevant (bool), confidence $\in [0,1]$, and reasoning (string). Malformed responses default to accept=True, confidence=0.5, so failures of the judge never silently block the system. It is also architecturally orthogonal; the Reviewer node does not care what specific generator it reviews. The confidence score is the average score from the following rubric dimensions (each scored 1-10): - Correctness (does the code actually fix the issue?) - Edge Cases (boundary conditions, null paths, race conditions) - Security (injection vectors, unsafe deserialisation) - Code Quality (readability, idiomatic style) - Relevance (directly addresses the question, not generic advice) Confidence is $C = \text{mean}(s_1..s_5) /$

10. If $C \geq 0.7$ accept; if $C < 0.7$ re-invoke the generator with the reasoning string as feedback. The loop is limited to three iterations.

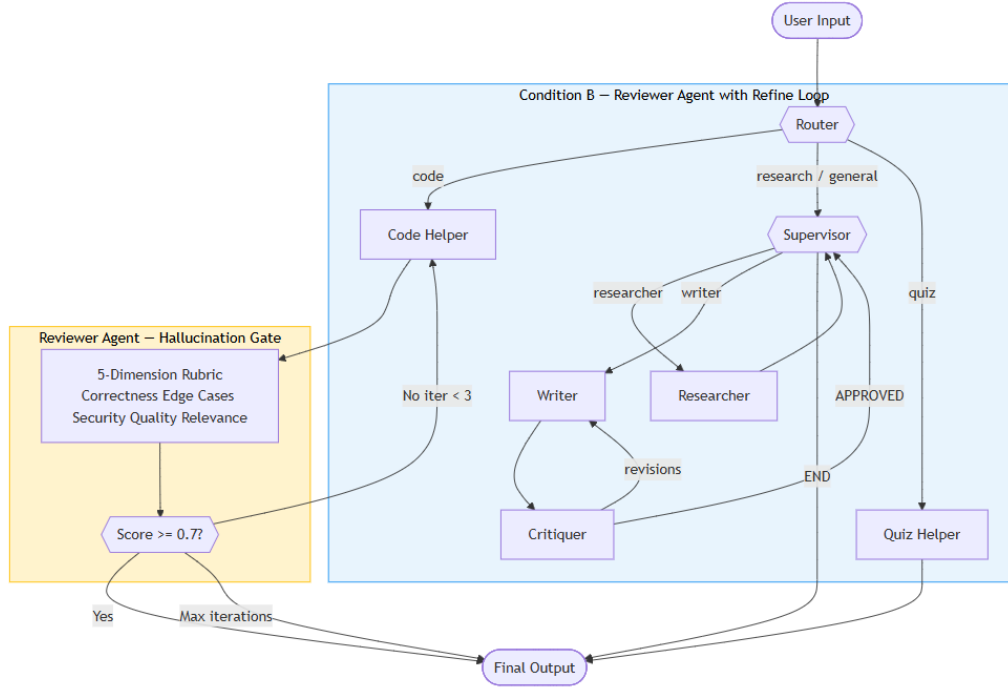


Figure 2. Condition B, Reviewer-gated refine loop. Outputs scoring below 0.7 are re-sent to the Code Helper with structured feedback; the loop iterates at most three times.

3.3 The Hallucination Bridge

Condition C replaces the Code Helper with OpenHands [5] generating pairs of ActionEvent and ObservationEvent at each agent step. A callback is registered to catch these events, trim their aggregate summaries to 1,500 characters each, and then save in ChatState as openhands_action_trace after serialising. The Reviewer does a check on this trace with two extra features: (i) a strong denial of a prior observation is assumed to be a hallucination signal, (ii) the judge does an explicit check to see if the agent actually ran the tests and observed the output instead of just claiming success. This is targeted at the usual failure case of fabricated test results in the agent's last messages.

ACTION TRACE • django_django-18914		REVIEWER FLAG
ACT	bash\$ grep -rn 'managers' django/db/	-
OBS	django/db/models/manager.py:142 class Manager...	-
ACT	edit django/db/models/manager.py L142-168	-
OBS	wrote 4 lines, removed 2 lines	-
ACT	claim: "I added test_managers.py"	🚩 HALLUC
OBS	ls tests/ → no such file	🚩 HALLUC

Figure 3. Condition C, OpenHands action trace captured by the Hallucination Bridge. Each ACT/OBS pair is recorded; contradictions between claimed and observed outcomes are flagged.

3.4 Experimental Design

Five controlled sweeps were performed on 19 SWE-bench Lite instances, distributed across four repositories (django, sympy, scikit-learn, astropy). These 19 instances were used for all sweeps. Table 1 shows the details of the sweep parameters.

Sweep	Purpose	Conditions	Generator / Judge
sweep_phase1	Phase 1 Baseline ($\tau=0.70$)	A / B / C	qwen3-32b / qwen3-32b
sweep_thresh85_b	Threshold Sensitivity B ($\tau=0.85$)	B only	qwen3-32b / qwen3-32b
sweep_thresh85_c	Threshold Sensitivity C ($\tau=0.85$)	C only	qwen3-32b / qwen3-32b
sweep_judge_sep_a	Judge Separation A	A only	qwen2.5-7b / qwen3-32b
sweep_judge_sep_b	Judge Separation B ($\tau=0.85$)	B only	qwen2.5-7b / qwen3-32b

Table 1. Five experimental sweeps. τ = acceptance threshold.

4. Results

4.1 Phase 1 Baseline: Conditions A, B, C at Threshold 0.70

Table 2 shows the aggregate statistics across all 19 instances for the Phase 1 sweep. All three conditions used the same qwen3-32b model for both generation and judging.

Condition	Mean	σ	Min	Max
A, Baseline ($\tau=0.70$)	0.878	0.145	0.30	0.96
B, Reviewer+Refine ($\tau=0.70$)	0.881	0.069	0.72	1.00
C, Hallucination Bridge ($\tau=0.70$)	0.887	0.126	0.40	1.00

Table 2. Phase 1 results, 19 instances, threshold 0.70, same model for generator and judge.

At the 0.70 threshold, the mean difference in scores between A and B is only 0.003, but the difference in variances is quite large: B's standard deviation is less than half that of A's (0.069 vs 0.145). The Reviewer mainly decreases the score distribution without large effects on the mean as most samples are already above 0.70 at the first attempt. The minimum score also goes from 0.30 to 0.72 so no case leaves the loop with a very low score. Condition C adds a small mean gain (0.887) but with higher variance (0.126), the action-agent architecture performs better in some instances and worse in others.

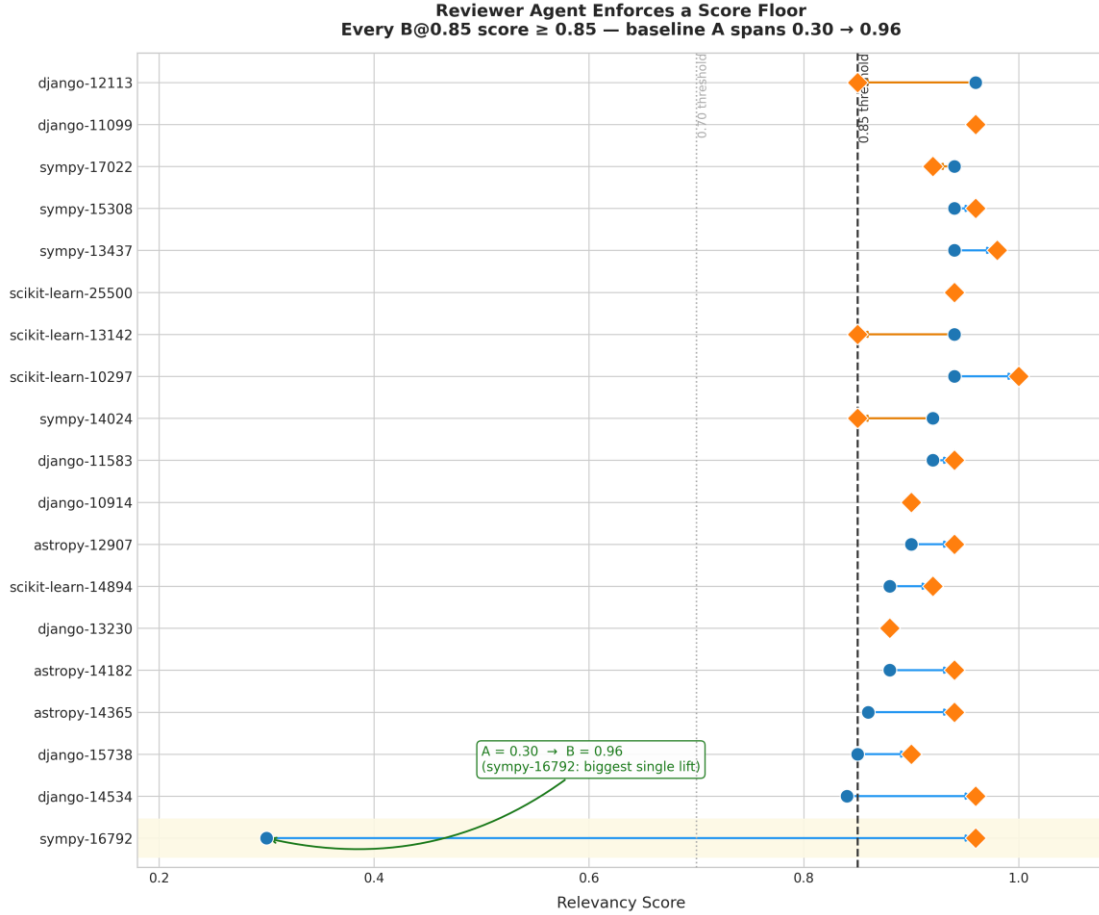


Figure 4. Per-instance scores: Condition A (baseline) versus Condition B@0.85. Green arrows show the $A \rightarrow B$ lift; the dashed line marks the 0.85 acceptance threshold. *sympy-16792* shows the largest single-instance improvement (0.30 $\rightarrow$ 0.96).

4.2 Threshold Sensitivity: The Score Floor Effect

At threshold 0.70 the difference in score between A and B was only 0.003. But the difference in variance was huge. B's standard deviation was less than half of A's (0.069 vs 0.145). The Reviewer compresses the score distribution without big mean shift as most instances are already associating the 0.70 threshold at the first try. The minimum score is also improved from 0.30 to 0.72, which means no instance would be out of loop with very low score.

Adding Condition C gives a slightly higher average score (0.887) but also a higher variance (0.126) as well. The action-agent model is better on some instances, but it is also worse on others. Raising the threshold to 0.85 strongly affects Condition B. The mean score increases from 0.881 to 0.926, the standard deviation decreases from 0.069 to 0.044 and the minimum score is only 0.85 i.e. no instance leaves the refine loop below the acceptance threshold. The key empirical observation is that the refine loop creates a hard score floor equal to the threshold value.

sympy-16792 is a good illustration of the mechanism at work. The baseline scored it 0.30, the lowest score in the dataset. B@0.85 The first response was not accepted by the reviewer and three refinement loop changes were pointed out. Finally the response was accepted at 0.96. The structured feedback from the Reviewer took each iteration closer towards a detailed diagnosis of the bug in the separability matrix. That first 0.30 would have been the final score had it not been for the Reviewer.

Of the total of 19, 14 (73%) showed score increases with B@0.85 over A. In 5 cases, the score drop was really small (avg. loss of ~ 0.03 points) and all 5 of them had a score above 0.85, so the drop got compensated by the floor guarantee.

Condition / Threshold	Mean	σ	Min
A ($\tau=0.70$)	0.878	0.145	0.30
B ($\tau=0.70$)	0.881	0.069	0.72
B ($\tau=0.85$)	0.926	0.044	0.85

Table 3. Effect of threshold on Condition B. Raising τ from 0.70 to 0.85 enforces a hard floor and halves the standard deviation.

4.3 Condition C at Threshold 0.85: Bimodal Distribution

Unlike Condition B, increasing the threshold for Condition C from 0.70 to 0.85 results in total loss. The mean score drops from 0.887 to 0.721 and the standard deviation almost triples ($0.126 \rightarrow 0.353$). The graph exhibits a two-peak pattern with 14 cases strongly clustered in the range of 0.86–1.0, and 5 cases are below 0.40, with one even 0.00.

The primary reason is context explosion. OpenHands gathers roughly 150,000 tokens at the 30th step of a complex instance. At this level the judge is confused and accepts even answers opposite to trace or scores based on length of action sequence rather than correctness. Trying to force more refinement iterations with a tighter threshold only extends the already long traces, and thus increases confusion. This situation is usually safe for the threshold of 0.70, because the first OpenHands trial usually provides a good enough answer on bugs that can be solved.

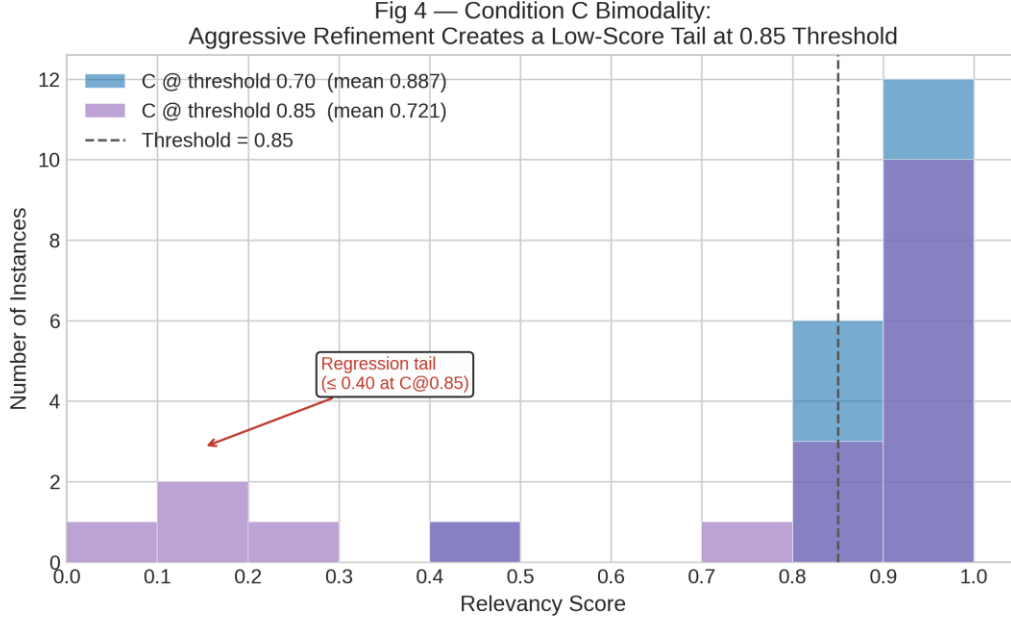


Figure 5. Score distribution for Condition C at threshold 0.70 versus 0.85. C@0.85 is bimodal: 14 instances score above 0.85 while 5 instances fall below 0.40.

4.4 Judge Separation: Reviewer Value Scales with Generator Weakness

For the Phase 1 sweeps, the model used for both generation and evaluation was the qwen3-32b. The role of The Reviewer was truly determined with two more sweeps with qwen2.5-7b as the generator and qwen3-32b as the critic.

When the generator and the judge were both the same-model and strong, the Reviewer gives $\Delta = +0.003$ over baseline ($0.878 \rightarrow 0.881$). With the weak generator, the baseline mean is 0.657 and the Reviewer makes it 0.771, a $\Delta = +0.114$ improvement. This suggests that the Reviewer is needed even when the generator is very good, and acts as a fall-back whose importance increases as the generator’s guesses become more uncertain. The greater the gap between the quality of the generator and the acceptance threshold, the more iterations of refinement get triggered and the more value the Reviewer’s structured feedback has.

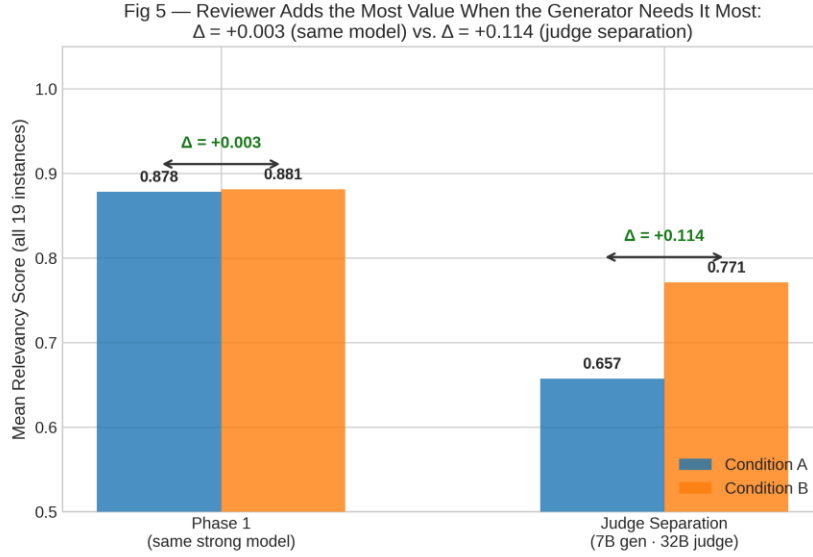


Figure 6. Judge separation results. Left: Phase 1 (same qwen3-32b model), $\Delta = +0.003$. Right: Judge separation (qwen2.5-7b generator, qwen3-32b judge), $\Delta = +0.114$. The Reviewer contributes most where the generator is weakest.

Sweep	Condition	Mean	σ	Δ vs A
Phase 1 (same model)	A	0.878	0.145	—
Phase 1 (same model)	B @ $\tau=0.70$	0.881	0.069	+0.003
Phase 1 (same model)	B @ $\tau=0.85$	0.926	0.044	+0.048
Judge sep (7b gen)	A	0.657	0.270	—
Judge sep (7b gen)	B @ $\tau=0.85$	0.771	0.218	+0.114

Table 4. Full results summary across all sweeps.

5. Discussion

5.1 The Refine Loop as a Score Floor

The most consistent observation over all the runs is that the Condition B refine loop acts like a hard score floor: the lowest score in any B run is always at least the acceptance threshold. This is a direct consequence of the algorithm's design, a score less than the threshold leads to re-generation, and the loop only breaks when the threshold is met or the maximum number of iterations is reached. Actually, in most cases, the maximum number of iterations was not reached: the majority of cases that initially scored below the threshold were accepted within one or two iterations. The floor characteristic really means that the average score can be deceiving: under Condition B no player gets a response that the Reviewer decided as failing, irrespective of how the initial generation was.

5.2 Threshold Calibration

The choice of the acceptable threshold level is, by far, the major determinant of the system. At Conditions B, there is always a higher threshold, which is good because it makes the system go through more refinements and lifts both average and floor scores without any new failure modes, as the Code Helper still produces nice, context-bounded replies that don't suffer from length explosion. However, at Conditions C, it is quite the opposite beyond 0.80: higher thresholds lead to more iterations of an already long-context agentic run, eventually compromising the judge's coherence. Therefore, each architecture should have its own optimal threshold setting rather than having a single global constant. Adaptive threshold calibration, which varies the acceptance boundary depending on the estimated context length, is the next logical step.

5.3 Reviewer Value and Generator Quality

The judge separation experiment aims to answer a question that the data from Phase 1 could not: does the Reviewer add any value when the generator is already strong, or does its effect simply come from working with a high quality model alone? The value of Δ in the weak generator case is +0.114, and in the strong generator case is +0.003, as the data shows. The Reviewer is not merely checking the results of a very capable model, it is indeed finding and fixing failures the weaker generators missed. This will matter for real deployment: entities with lightweight models running on inference-constrained hardware will indeed get most of their improvements from the Reviewer, which was limited to their weakest points [12].

5.4 Limitations

The present findings cannot be extended because of certain limitations. The 19-instance evaluation gives us a directional signal, but is limited in its power to detect difference, and the whole SWE-bench Lite run (300 instances) is the clear next step. The free-tier hosted LLM exhibited some non deterministic latency variance due to shared rate limiting and a small fraction of Condition C runs were incomplete due to transient network error. Absolute relevance scores are highly dependent on the judge model; they can be shifted uniformly simply by changing the critic model, which makes cross-paper comparisons of raw confidence values difficult without the same judge configuration.

6. Conclusion

The article talked about a lightweight, prompt-based hallucination detection system that the Reviewer Agent uses inside a LangGraph multi-agent implementation. In five sweeps across 19 instances of the SWE-bench Lite test set, the main finding is that a refine loop driven by a score actually sets a hard score floor equal to the acceptance threshold, not a single instance comes out of the loop below the threshold. At threshold 0.85 the Reviewer is able to lift the mean relevancy from 0.878 to 0.926 while simultaneously reducing the standard deviation from 0.145 to 0.044. The Hallucination Bridge turns this audit tool into the Analysis method for the per-step action traces of an autonomous coding agent, so that it can produce forensically verifiable evidence for each agent action, and this neither requires fine-tuning nor logit access.

The judge separation experiment reveals the Reviewer's biggest point of contribution exactly where it is most needed: weak generators obtain a $\Delta = +0.114$, strong generators get $\Delta = +0.003$. This indicates that the Reviewer could be thought of as an error-correction layer whose usefulness is inversely related to generator quality. The bimodal Condition C failure at high thresholds draws attention to context length being a first-rate problem for action-agent review, hence the motivation for adaptive threshold strategies.

References

- [1] Jimenez C.E. et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. arXiv:2310.06770.
- [2] Ouyang L. et al. Training Language Models to Follow Instructions with Human Feedback. NeurIPS, 2022. arXiv:2203.02155.
- [3] Manakul P., Liusie A., Gales M.J.F. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative LLMs. EMNLP, 2023. arXiv:2303.08896.
- [4] Huang L. et al. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges. arXiv:2311.05232, 2023.
- [5] Wang X. et al. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. arXiv:2407.16741, 2024.
- [6] Maynez J. et al. On Faithfulness and Factuality in Abstractive Summarization. ACL, 2020. arXiv:2005.00661.
- [7] Ji Z. et al. Survey of Hallucination in Natural Language Generation. ACM Computing Surveys, 2023. arXiv:2202.03629.
- [8] Chen M. et al. Evaluating Large Language Models Trained on Code (Codex). arXiv:2107.03374, 2021.
- [9] Zheng L. et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. NeurIPS, 2023. arXiv:2306.05685.
- [10] Madaan A. et al. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS, 2023. arXiv:2303.17651.
- [11] Shinn N. et al. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS, 2023. arXiv:2303.11366.
- [12] Pan L. et al. Automatically Correcting Large Language Models: Surveying the Landscape of Diverse Self-Correction Strategies. TACL, 2024. arXiv:2308.03188.